

· [KLDP.org](#) · [KLDP Wiki](#) · [KLDP.net](#) · [통합 검색](#) ·



[Subversion Book/Advanced Topics](#)

[FrontPage](#) | [FindPage](#) | [TitleIndex](#) | [RecentChanges](#) |

[SubversionBook/Introduction](#) › [SubversionBook/GuidedTour](#) ›
[SubversionBook/BasicConcepts](#) › [SubversionBook/BranchingAndMerging](#) ›
[SubversionBook/RepositoryAdministration](#) › [SubversionBook/AdvancedTopics](#)

Login:

Password:

[Join](#)

He who has a shady past knows
that nice guys finish last.

[madkid](#)
[ProgrammersAtWork](#)
[DocbookSgml/SysAdminGuide-TRANS](#)
[UserInterfaceEngineering](#)

1Chapter. 고급 주제

Table of Contents

- 1.1.
- 1.1. [실행시 설정 area](#)
 - 1.1.1. [설정 area의 레이아웃](#)
 - 1.1.2. [설정과 Windows의 레지스트리](#)
 - 1.1.3. [설정 옵션](#)
 - 1.1.3.1. [서버](#)
 - 1.1.3.2. [Config](#)
- 1.2. [속성](#)
 - 1.2.1. [왜 속성은 물건이?](#)
 - 1.2.2. [속성의 조작](#)
 - 1.2.3. [특수한 속성](#)
 - 1.2.3.1. [svn:executable](#)
 - 1.2.3.2. [svn:mime-type](#)
 - 1.2.3.3. [svn:ignore](#)
 - 1.2.3.4. [svn:keywords](#)
 - 1.2.3.5. [svn:eol-style](#)
 - 1.2.3.6. [svn:externals](#)
- 1.3. [외부 정의](#)
- 1.4. [벤더 브랜치\(branch\)](#)
 - 1.4.1. [일반적인, 벤더 브랜치\(branch\)를 관리하는 방법](#)
 - 1.4.2. [svn-load-dirs.pl](#)



kss@kldp.org

1.1.

만약, 이 책을 장 마다 최초부터 끝까지 읽고 있다면, 이제(벌써) 당신은 대부분의 버전 관리 조작을 실행하기 위해서 Subversion 클라이언트를 사용하기 위한 충분한 지식을 가지고 있을 것입니다. 어떻게 Subversion 저장소(repository)로부터 작업 카피를 체크아웃 하는지를 이해하고 있을 것입니다. **svn commit** 나 **svn update**를 사용한 변경점의 송수신에 친숙해져 있다는 입니다. 그리고 아마 거의 무의식 중에 **svn status** 를 실행해 버리는 것 같은 반사 신경마저 몸에 붙어 있을지도 모릅니다. 어떤 의도나 목적에 대해서도, 전형적인 환경에서 Subversion을 사용할 준비가 되어 있을 것입니다.

그러나, Subversion의 기능 세트는 "보통 버전 관리 조작"으로 끝나는 것은 아닙니다.

이 장에서는 몇개의 Subversion의 기능으로, 그만큼 빈번하게는 이용되지 않는다 같은 것을 취합니다. 그 안에서, Subversion의 속성(혹은 "메타데이터")의 서포트에 대해 논의해, 어떻게 해 Subversion의 디폴트의 행동을 실행시 설정 area의 조정에 의해 변경 할 수가 있을까를 봅니다. 또, 어떻게 외부 정의를 사용해, 복수의 저장소(repository)로부터 데이터를 끌어오도록(듯이) Subversion에 인스트럭션 하는지를 설명합니다. 그리고, Subversion의 패키지 지의 일부인 추가의 클라이언트측, 서버가 원의 툴의 몇개의 상세하게도 접합니다.

이 장을 읽기 전에, Subversion로 기본적인 파일과 디렉토리에 관한 버전 관리의 능력에 대해 친숙해져 있어야 합니다. 만약 아직 거기에 붙어 (은)는 읽지 않은지, 복습이 필요하면, >이라고 > (을)를 읽는 것을 추천합니다. 한 번 기본을 마스터 하고 나서 이 장을 소화하면, 당신은 이제 Subversion 의 파워 유저입니다혹은, 돈은 답례합니다! [\[1\]](#)

1.1. 실행시 설정 area

Subversion은 많은 옵션의 행동을 준비해 있어, 그것은 유저에게 따라서 제어할 수가 있습니다. 그러한 옵션의 상당수는 유저가 모든 Subversion 조작에 적용하고 싶다고 생각하는 것 같은 일입니다. 그래서, 이러한 옵션 (을)를 지정하기 위해서 유저에게 커멘트 라인 인수를 생각나게 하도록(듯이) 강요하는 것보다도 또, 실행하려고 하는 모든 조작에 대해서 그것들을 사용하는 것보다도, Subversion은 정의 파일을 사용합니다. 그것은 Subversion의 정의 area 로 분리되고 있는 것입니다.

Subversion의 설정 area 는 2층으로 나누어진 옵션명과 값의 계층입니다. 보통, 이것은 정의 파일 (최초의 층)을 포함한 특별한 디렉토리에 요약되어 있어, 그것은 표준적인 INI 형식의 텍스트 파일에 지나지 않습니다. (거기에는 "sections" 가 있어, 그것이 제2층이 됩니다) 이러한 파일은 좋아하는 텍스트 문자 편집기를 사용해 간단하게 편집할 수가 있습니다. (emacs 라든지 vi 라든지) 그리고, 클라이언트 에 의해 읽히는 인스트럭션을 포함하고 있어, 유저가 좋아하는 다양한 옵션의 행동을 어떻게 할까를 결정합니다.

1.1.1. 설정 area의 레이아웃

svn 커멘트 라인 클라이언트가 최초로 실행되면(자), 그것은 유저마다의 구성 area를 만듭니다. Unix풍의 시스템이라면, 이 area는 유저의 홈 디렉토에, .. subversion 라는 이름의 디렉토리로 준비됩니다. Win32 시스템에서는, Subversion은 Subversion라는 이름의 폴더를 만듭니다. 보통으로는 유저 프로파일 디렉토리의 Application Data area 의 내부가 됩니다. 그러나, 이 플랫폼에서는, 완전한 장소는 시스템 마다 차이가 나, 진짜 장소는 Windows 레지스트리로 설정되어 있습니다. 유저마다의 설정 area는, Unix 에서의 이름이다. subversion (을)를 사용해 참조하기로 하겠습니다.

유저마다의 설정 area에 가세해, Subversion은 시스템 전체의 설정 area도 이해할 수가 있습니다. 이것은 시스템 관리 책임자에 있는 머신상에서의 모든 유저에 대한 디폴트를 설정하는 힘을 줍니다. 시스템 전체 의 설정 area는 필수의 폴리스가 있는 것은 아닙니다. 유저 마다의 설정 area는, 시스템 전체의 area를 덮쓰기해, svn 프로그램에게 주는 커멘트 라인 인수는 행동을 결정하는 마지막 장소에 됩니다. Unix풍의 플랫폼에서는, 시스템 전체의 설정 area는 /etc/subversion 디렉토리에 있다고 기대되어 있습니다. 그것은 또, 공통 어플리케이션 area의 내부에 있다 Subversion 디렉토리를 보러 갑니다. (그리고 또, 그것은 Windows 레지스트리에 의해 지정됩니다만) 유저마다의 경우 (와) 달리, svn 프로그램은 시스템 전체의 설정 area를 만들려고는 하지 않습니다.

. subversion 디렉토리는 현재로서는 세 개의 파일을 포함하고 있습니다. 두 개의 설정 파일 (config 와 servers입니다), 거기에 README.txt 파일로, 이것은 INI 형식을 설명하는 것입니다. 그러한 생성시에는, 파일은 Subversion 하지만 서포트하는 각각의 옵션의 디폴트값이 들어오고 있어 대부분이 comment out 되고 있어, 게다가 어떻게 키에 대한다 값이 Subversion의 행동에 영향을 줄까라고 하는 것에 대하여, 텍스트의 설명 돌출하고 그룹화되고 있습니다. 무엇인가의 행동을 바꾸기 위해서(때문에)는 관련하는 설정 파일을 텍스트 문자 편집기로 열려, 필요한 옵션치로 수정하는 것만이 필요합니다. 만약 항상 설정 파일중에 디폴트의 설정 (을)를 하고 싶은 경우는, 단지 그 파일을 삭제해, 무엇인가 무해인 svn 개만, 예를 들어 svn --version의 같은 것을 실행하면(자), 없어진 파일은 디폴트 상태로 재생성됩니다.

유저마다의 설정 area는 인증 데이터의 캐쉬도 포함합니다. auth 디렉토리는 Subversion로 서포트되고 있는 다양한 인증 방법으로 이용된다 캐쉬 정보의 요소를 포함한 서브 디렉토리의 모임을 보관 유지합니다. 이 디렉토리는 유저 자신만이 그 내용을 읽을 수가 있는 것 같은 형태에 작성됩니다.

1.1.2. 설정과 Windows의 레지스트리

보통 INI 베이스의 설정 area에 가세해, Windows 플랫폼상에서 실행되어 있는 Subversion 클라이언트는 Windows의 레지스트리도 설정 데이터를 격납하는 장소 (으)로서 이용할 수가 있습니다. 옵션명과 그 값은 INI 파일중과 같다 입니다. "file/section" 의 계층 관계도 보존됩니다. 조금 다르다 방법에 따릅니다만 이 방법에서는, 파일과 섹션은 단지 레지스트리 키의 트리의 계층 밖에 지나지 않습니다.

Subversion은 시스템 전체의 설정치를

HKEY_LOCAL_MACHINE\Software\Tigris.org\Subversion 키 의 원으로 검색합니다. 예를 들어 global-ignores 옵션, 이것은 config 파일의 miscellany 섹션에 있습니다만, HKEY_LOCAL_MACHINE\Software\Tigris.org\Subversion\Config\Miscellany\Global-ignores 에 찾아낼 수가 있습니다. 유저마다의 설정치는 HKEY_CURRENT_USER\Software\Tigris.org\Subversion. 의 아래에 격납될 것입니다.

레지스트리 베이스의 설정 옵션은, 파일 베이스의 나머지의 부분을 검색하기 전에 검색됩니다. 그래서, 이러한 옵션은, 설정 파일중에서 발견된 값에 의해 덮쓰기 됩니다. 바꾸어 말하

면(자), 설정의 우선권은 Windows 시스템의 경우, 이하의 순서가 되는 것이 프로텍션되고 있습니다:

1. 커멘드 라인 옵션
2. 유저마다의 INI 파일
3. 유저마다의 레지스트리치
4. 시스템 전체의 INI 파일
5. 시스템 전체의 레지스트리치

또, Windows 레지스트리는 "comment out" (와)과 같은 개념을 서포트하고 있지 않습니다. 그러나, Subversion은, 키의 이름이 해시 캐릭터(#)로 시작되는 것 같은 모든 옵션을 무시합니다. 이것으로 실제로는 Subversion의 옵션 (을)를, 레지스트리로부터 완전하게 키를 지우지 않고 comment out 할 수가 있습니다. 분명하게, 그 옵션의 설정 작업은 간단하게 하고 있습니다.

svn 커멘드 라인 클라이언트는 Windows의 레지스트리에 기입하는 것은 결코 않고, 거기에 디폴트의 설정치를 만들려고도 하지 않습니다. 필요한 키는 **REGEDIT** 프로그램으로 만들 수가 있습니다. 다른 방법 (으)로서는, . REG 파일을 만들어, 익스플로러 조가비로부터 그 파일을 더블 클릭 하면(자), 그 데이터가 레지스트리에 merge 됩니다.

Example 1-1. 레지스트리 엔트리(. REG) 파일의 예

REGEDIT4

```
[HKEY_LOCAL_MACHINE\Software\Tigris.org\Subversion\Servers\DEFAULT]
"http-proxy-host"=""
"http-proxy-port"=""
"http-proxy-username"=""
"http-proxy-password"=""
"http-proxy-timeout"="0"
"http-compression"="yes"

[HKEY_CURRENT_USER\Software\Tigris.org\Subversion\Config\Wauth]
"store-password"="no"

[HKEY_CURRENT_USER\Software\Tigris.org\Subversion\Config\Whelpers]
"editor-cmd"="notepad"

[HKEY_CURRENT_USER\Software\Tigris.org\Subversion\Config\Wmiscellany]
"global-ignores"="*.o *.lo *.la ##. *.rej *.rej . *~ *~ . ##"
```

이 예는, . REG 의 내용을 나타낸 예입니다만, 그 안에는, 자주(잘) 이용되는 설정 옵션의 대부분 그 디폴트치가 있습니다. 시스템의 설정(예를 들어 네트워크 프록시에 관한 옵션) 라고 유저마다의 설정(이용하는 에디터, 패스워드, 등)의 양쪽 모두가 있는 것에 주의해 주세요. 한층 더 모든 옵션은, comment out 되고 있게도 주의해 주세요. 옵션명의 선두의 해시 캐릭터 (#)(을)를 없애는 것만으로, 바라고 있는 값으로 설정하는 것이 할 수 있습니다.

1.1.3. 설정 옵션

이 장에서는, 특정의 실행시 옵션에 대해 논의합니다. 현재 Subversion 하지만 서포트하고 있는 것에 임해서입니다.

1.1.3.1. 서버

servers 파일은 Subversion의 설정 옵션으로, 네트워크층에 관계한 것을 포함하고 있습니다. 두 개의 섹션명이 그 파일에는 있는 groups 와 global 입니다. groups 섹션은, 요컨대 크로스 레퍼런스의 테이블입니다. 이 섹션의 키는, 파일중에 있는 다른 섹션의 이름 입니다. 그 값은 그로브와일드 카드를 포함해 있을지도 모르는 텍스트 토큰입니다로, Subversion의 요구가 송신 되는 머신의 호스트 명칭이라고 비교됩니다.

[groups]

```
beanie-babies = *.red-bean.com
collabnet = svn.collab.net
```

```
[beanie-babies]
```

```
[collabnet]
```

Subversion가 네트워크 넘어로 이용되는 경우, groups 섹션에 있는 그룹명에 맞는 서버 명칭과 성냥 하는 것을 찾습니다. 만약 성냥 했을 경우는 Subversion은 다음에, 그 이름이 그룹 명칭과 성냥 했다 servers 파일중의 섹션을 찾습니다. 그리고 그 섹션으로부터 실제의 네트워크 설정 옵션을 읽어들입니다.

global 섹션은 groups 섹션 목의 그로브에도 들어맞지 않았다 모든 서버에 대한다 설정이 있습니다. 이 섹션으로 사용할 수 있는 옵션은, 파일의 다른 서버 섹션으로 이용할 수 있는 것과 완전히 같습니다. (다만, 물론, groups 섹션은 예외입니다) 이하와 같은 느낌입니다:

```
http-proxy-host
```

이것은, 프록시 컴퓨터의 호스트 명칭으로, HTTP 베이스의 Subversion (은)는 거기를 통해서 통신하지 않으면 안됩니다. 디폴트는 하늘에서, 그것은 Subversion은 프록시를 통해서 HTTP 요구하지 않고, 직접, 목적의 머신과 통신하려고 하는 것을 의미하고 있습니다.

```
http-proxy-port
```

이것은, 이용하는 프록시 호스트의 포트 번호를 지정합니다. 디폴트는 하늘입니다.

```
http-proxy-username
```

이것은, 프록시 머신으로 필요한 유저명을 지정합니다. 디폴트는 하늘입니다.

```
http-proxy-password
```

이것은, 프록시 머신으로 필요한 패스워드를 지정합니다. 디폴트는 하늘입니다.

```
http-timeout
```

이것은 서버 응답을 기다리는 시간의 최대치를 초단위로 지정합니다. 만약, Subversion의 조작이 타임 아웃 해 버리는 것 같은 저속의 네트워크 접속에 관계한 문제를 떠안고 있는 경우, 이 옵션의 값을 늘려 보세요. 디폴트치는 0으로, 이 경우, HTTP 프로그램 라이브러리가 다 Neon에 디폴트의 타임 아웃치를 사용하도록(듯이) 지시합니다.

```
http-compression
```

이것은, DAV가 유효한 서버로, Subversion가 네트워크 요구 데이터를 압축하는지 어떤지를 지정합니다. 디폴트치는 yes (다만, 압축은 네트워크층의 컴파일시에 유효하게 되지 않으면 되지 않습니다만,)입니다. no 로 설정하면(자) 압축은 무효가 됩니다만, 이것은 네트워크 전송의 디버그시 등에 사용합니다.

```
neon-debug-mask
```

이것은, 정수치의 마스크로, HTTP 프로그램 라이브러리 Neon 가 어떠한 타입의 debug 출력을 생성하는지를 지정하는 것입니다. 디폴트는 0으로, 모두 디버그 출력을 무효로 합니다. Subversion 하지만 Neon를 어떻게 사용할까에 대한 자세한 것은 >(을)를 봐 주세요.

```
ssl-authorities-file
```

이것은 HTTPS 경유로 저장소(repository)에 액세스 할 경우에 Subversion 클라이언트에 의해 받아들여지는 인증 기관(혹은 CA)의 증명서를 포함한 파일의 패스를 지정합니다.

```
svn-tunnel-agent
```

이것은 외부 에이전트 프로그램을 지정합니다. 이 프로그램 (을)를 통해서, SVN 프로토콜 요구가 터널 됩니다.

1.1.3.2. Config

config 파일은, Subversion 실행시 옵션 가운데, 현재 이용할 수 있는 나머지의 것으로, 네트워크에 관련하는 것 이외가 포함되어 있습니다. 현시점에서는 몇개의 옵션 하지만 이용할

수 있을 뿐입니다만, 향후의 추가를 생각해, 다른 섹션으로서 그룹화 차고라고 있습니다.

auth 섹션은 Subversion의 저장소(repository)에 대한 인증과 허가에 관계한 설정이 있습니다. 그것은:

store-password

이것은 Subversion에 서버 인증 챌린지에 대해서 유저가 입력하는 패스워드를 캐쉬하는지 여편지를 지시합니다. 디폴트는no 로, 디스크상에 패스워드 (을)를 캐쉬하지 않습니다. 이 옵션은**svn** 커멘트 그리고--no-auth-cache 를 사용하면(자) 덧쓰기할 수가 있습니다. (혹은 이 인수를 서포트하고 있는 커멘트이면 어떤 것에서도)

helpers 섹션은 Subversion가 어느 외부 어플리케이션을 몇개의 처리로 사용하는지를 제어합니다. 이 섹션으로 유효한 것은:

editor-cmd

이것은 Subversion가 커밋시의 로그 메시지를 만드는데 어느 프로그램을 사용하는지를 지정합니다. 예를 들어,**svn commit** 가, --message (-m)도 --file (-f) 옵션 도 없음으로 실행된 것 같은 경우입니다. 이 프로그램은 또**svn propedit**커멘트에서도 사용됩니다.일시적인 파일에 유저가 편집하고 싶다고 생각하는 현재의 속성치가 기입해집니다만, 이것은 에디터의 기동에 의해 실행됩니다. (> 참조). 이 옵션은 디폴트는 하늘입니다. 만약 이 옵션이 설정 되어 있지 않으면 Subversion은 환경 변수 SVN_EDITOR, VISUAL, 라고 EDITOR (이 순서로) (을)를 조사합니다.

diff-cmd

이것은 차분 표시 프로그램의 절대 패스를 지정합니다. 이 프로그램은 Subversion가 "diff"의 출력을 생성하는데 이용 되는 것입니다(**svn diff** 커멘트 실행시 등입니다). 디폴트치는 GNU diff 유틸리티의 패스로, 그것은 Subversion의 원시 코드 구축 시스템에 의해 결정됩니다.

diff3-cmd

이것은 스리웨이 차분 프로그램의 절대 패스를 지정합니다. Subversion (은)는 이 프로그램을 저장소(repository)로부터 받은, 유저가 한 변경점을 merge 하는데 사용합니다. 디폴트치는 GNU diff3 유틸리티의 패스로, 이것은 Subversion의 원시 코드 구축 시스템에 의해 결정 됩니다.

diff3-has-program-arg

이 플래그는diff3-cmd 옵션이 --diff-program 파라미터를 받아들이는 경우에는 true를 지정해야 합니다. diff3-cmd 옵션의 디폴트치는 컴파일시에 결정할 수 있으므로, diff3-has-program-arg의 디폴트치도 그렇습니다.

miscellany 섹션은 다른 장소에 둘 수 없다 모든 것의 두는 곳소입니다. [\[2\]](#) 이 섹션에는:

global-ignores

svn status커멘트를 실행하면(자) Subversion은 버전화 되지 않는 파일과 디렉토리를 버전화 되어 있는 것과 함께 일람표 가리킵니다. 이 때 버전화되어 있지 않다 일을? 캐릭터로 표현합니다. (>참조). 가끔, 그다지 흥미가 없는 버전화 되지 않는 아이템이 표시되는 것을 보는 것을 귀찮게 생각하는 일이 있습니다. 예를 들어, 프로그램의 컴파일에 의해 할 수 있는 오브젝트 파일 등 global-ignores 옵션은 공백에서 단락지어진 그로브의 리스트로, 버전화되어 있지 않은 것이면 Subversion에 표시해 주었으면 하고 없는 이름의 지정이 됩니다. 디폴트는 *. o *. lo *. la ### . *. rej *. rej . *~ *~ . #*입니다.

svn status 커멘드로--no-ignore 플래그를 사용하는 곳의 옵션을 그 실행에 한해서 덧쓰기할 수 있습니다. 무시하는 아이템의, 좀 더 세세한 제어에 대해서는 >(을)를 봐주세요.

1.2. 속성

벌써, Subversion가 어떻게 해 저장소(repository)중에 있는 파르이나 디렉토리의 여러가지 버전을 격납해, 추출할까를 자세하게 봐 왔습니다. 모든 장은 Subversion라고 하는 틀에 의해 제공되고 있는 이 1번 기본적인 기능에 바칠 수 있어 왔습니다. 그리고, 만약 버전 관리의 서포트가 그래서 마지막이라면 해도, Subversion은 버전 관리의 관점으로부터는 완전한 것이었지라고 생각합니다. 그러나 이야기에게는 아직 앞이 있습니다.

디렉토리와 파일의 버전 관리 고기원네라고, Subversion은 버전화 된 파일, 디렉토리에 부수 한 버전화 되었다 메타데이터의 추가, 수정, 삭제에 위한 인터페이스를 준비해 있습니다. 이러한 메타데이터를 속성이라고 부릅니다. 속성은 작업 카피중의 아이템 마다, 이름과 이름에 결합된 임의의 값의 조로부터 되는 두 개의 열을 가지는 테이블로서 생각할 수가 있습니다. 일반적으로, 이름이 인간이 읽을 수 있는 텍스트가 아니면 안 되는 것을 들여다 보면, 이름과 속성치는 자유롭게 선택할 수가 있습니다. 그리고 속성에 관한 1번 중요한 (일)것은, 속성도 또, 파일의 내용과 같게 버전 관리할 수 있다 그렇다고 하는 것입니다. 텍스트의 변경점을 커밋하는 것과 같은 정도 간단하게 속성의 변경을, 수정하거나 커밋하거나 취소하거나 할 수가 있습니다. 그리고, 작업 카피를 갱신할 경우에, 다른 사람이 한 속성 변경에 대해서도 받을 수가 있습니다.

Subversion에서의 다른 속성

속성은 Subversion외의 장소에도 나옵니다. 파일이나 디렉토리가 거기에 결합된 임의의 속성명과 속성치를 가질 수 있는 것과 같이, 있다 리비전은, 그 자체로서 임의의 속성을 가질 수가 있습니다. 같은 제약이 들어맞습니다 속성명은 인간이 읽을 수 있는 텍스트로, 속성치는 바이너리치를 포함한 임의치로 다만, 리비전의 속성은 버전화 되지 않습니다. 버전화 되지 않는 속성에 대해서는 > (을)를 봐 주세요.

이 마디에서는, 속성을 서포트하는 유틸리티에 대해 설명하는 Subversion의 유저와 Subversion 그 자체에 대한 설명이 됩니다. 속성에 관련한 **svn** 서브 커멘드를 이해해, 속성의 변경이 통상의 Subversion의 워크플로우에 어떻게 영향을 줄까를 배웁니다. Subversion의 속성은 당신의 버전 관리의 경험을 넓히는 것인 것이, 반드시 알겠지요.

1.2.1. 왜 속성은 물건이?

속성은 작업 카피에 매우 도움이 되는 정보를 추가할 수가 있습니다. 실제, Subversion 자신도 특수한 정보를 기록하는데 속성을 사용하고 있어, 그것은 어느 특정의 처리가 필요하게 되어 있는 것을 나타내는 것 같은 때에 사용하고 있습니다. 같이 유저는 자기 자신의 목적을 위해서(때문에)도 속성을 사용한다 일이 생깁니다. 물론 속성으로 완성되는 것은 모두, 버전화 한 파일에서도 할 수 있습니다만, 우선은 이하와 같은 Subversion 속성의 사용법의 예를 봐 주세요.

아선반은, 많은 디지털 사진을 보이기 위한 웹 사이트를 설계하고 있어, 타이틀과 일자를 붙여 표시하고 싶다고 합니다. 여기서, 사진의 내용은 항상 변화하므로, 이 사이트의 관리를 할 수 있는 한 자동화하고 싶다고 생각하고 있습니다. 각각의 사진은 매우 크기 때문에, 이러한 경우의 상투수단으로서 당신은 사이트를 온 사람에게 작은 엄지손가락의 화상을 준비하고 싶다고 합니다. 이것을 보통 파일로 할 수도 있습니다. 즉, 디렉토리에 image123.jpg 와 image123-thumbnail.jpg 의 양쪽 모두를 두면 좋습니다. 혹은 양쪽 모두의 파일명칭을 함께 해, 별디렉토리에 있어도 괜찮네요. thumbnails/image123.jpg와 같은 느낌입니다. 타이틀과 일자에 대해서도 같은 방법을 취할 수가 있어 이것도 또, 원래의 화상 파일과는 다른 물건이 됩니다. 곧, 파일의 트리는 뒤죽박죽(이)가 되어, 새로운 사진이 사이트에 추가될 때마다, 사이트의 데이터 몇배에도 부풀어 오릅니다.

Subversion의 파일 속성을 사용한 같은 설정을 생각해 봅시다. 어느 화상 파일 image123.jpg 와 그 파일의 속성으로서 설정하는 caption, datestamp, 그리고 thumbnail가 있는 곳(중)을 상상해 주세요. 이렇게 스치고는, 작업 카피의 디렉토리는 좀 더 관리하기 쉽고 됩니다 실제 이것으로 화상 파일 이외가 아무것도 없게 보입니다. 그러나, 당신의 자동 스크립트는 좀 더 많은 일을 알고 있습니다. 그것은 **svn** (혹은 게다가 Subversion 언어 제휴를 사용할 수도 있는 > 참조)를 사용해 확장 정보를 추가합니다만, 그것은 당신의 사이트가, 인덱스 파일을 읽거나 복잡한 파일 패스 조작의 구조를 만지는 것 없이, 표시할 필요가 있다 물건입니다.

Subversion의 속성을 어떻게 사용할까는 당신 하기 나름입니다. 벌써 지적했다 같게, Subversion은 자기 자신이 사용하는 속성을 가지고 있어, 이 장의 나중에 조금 설명합니다. 그러나, 우선은, **svn** 프로그램 (을)를 사용해, 어떻게 옵션을 조작할까를 생각합시다.

1.2.2. 속성의 조작

svn 커멘드에는 파일과 디렉토리의 속성 (을)를 추가하거나 수정하거나 하는 몇개의 방법이 있습니다. 짧은 가독인 속성을 신규에 추가하는 1번 간단한 방법은 속성의 이름과 값을 **propset** 서브 커멘드로 지정하는 것입니다.

```
$ svn propset copyright '(c) 2003 Red-Bean Software' calc/button.c
property 'copyright' set on 'calc/button.c'
```

```
$
```

그러나, 속성치에 대해서 Subversion가 가지는 유연성에 대해서는 벌써 실컷 말해 왔습니다. 만약, 복수행 텍스트, 또는 바이너리치를 속성치에 하고 싶다고 생각하고 있다면, 커멘드 라인으로부터 그 값을 입력하고 싶지는 않으면 생각합니다. 그래서 **propset** 서브 커멘드는 --file(-F) 옵션을 사용해, 새로운 속성값이 들어왔다 파일의 이름을 지정할 수도 있습니다.

```
$ svn propset license -F /path/to/LICENSE calc/button.c
property 'license' set on 'calc/button.c'
$
```

propset 커멘드 고기원네라고, **svn** 프로그램은 **propedit** 커멘드도 준비해 있습니다. 이 커멘드는, 설정된 에디터를 사용해(>참조) 속성을 추가하거나 수정하거나 합니다. 이 커멘드를 실행하면(자)**svn** 는 현재의 속성치를 기입했다 일시파일을 만들어 에디터를 기동합니다. (새로운 속성을 추가하는 경우는 이것은 비웁니다). 그리고, 자신이 바라는 것 같은 값이 될 때까지 새로운 속성치 (을)를 에디터를 사용해 수정해, 일시파일을 보존하고 나서 에디터를 빠집니다. Subversion은 속성의 값이 변경된 것을 확인하면(자), 그것을 새로운 속성치 (으)로서 받아들입니다. 만약 에디터를 변경하는 일 없이 빠지면, 속성치의 변경은 일어나지 않습니다.

```
$ svn propedit copyright calc/button.c ### exit the editor without changes
No changes to property 'copyright' on 'calc/button.c'
$
```

다른 **svn** 커멘드와 같게, 속성에 관한 이러한 커멘드도 복수 패스에 대해서 한 번에 실행할 수가 있습니다. 이것은 하나의 커멘드로 복수의 파일상의 속성을 수정하는 것을 가능하게 합니다. 예를 들어, 이하와 같은 일이 생깁니다:

```
$ svn propset copyright '(c) 2002 Red-Bean Software' calc/*
property 'copyright' set on 'calc/Makefile'
property 'copyright' set on 'calc/button.c'
property 'copyright' set on 'calc/integer.c'

$
```

이러한 속성의 추가나 편집은, 보관되고 있는 속성치를 간단하게 취득 할 수 없으면, 별로 편리하지는 않습니다. 그래서 **svn** 프로그램은 파일이나 디렉토리에 보관되었다 속성의 이름과 값을 표시하기 위한 서브 커멘드를 들준비해 있습니다. **svn proplist**는 패스상에 존재하는 속성의 이름의 일람을 표시합니다. 노드상의 속성명을 알 수 있어 버리면, 개별적으로 **svn propget**를 호출해 그 속성치를 요구하는 것이 할 수 있습니다. 이 커멘드는 주어진(하나 이상의) 패스와 프롭퍼티명 (으)로부터, 그 속성치를 표준 출력에 표시합니다.

```
$ svn proplist calc/button.c
Properties on 'calc/button.c':
  copyright
  license
$ svn propget copyright calc/button.c
(c) 2003 Red-Bean Software
```

proplist 커멘드의 변종으로서 모든 속성의 이름과 값의 양쪽 모두를 리스트 하는 것이 있습니다. 이것에는 단지, --verbose(-v) 옵션을 지정하면 OK입니다.

```
$ svn proplist --verbose calc/button.c
Properties on 'calc/button.c':
  copyright : (c) 2003 Red-Bean Software
  license   : =====
Copyright (c) 2003 Red-Bean Software.  All rights reserved.
```

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions

are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions, and the recipe for Fitz's famous red-beans-and-rice.

마지막 프롭퍼티 관련 서브 커멘드는 **propdel**입니다. Subversion은 하늘의 값을 가지는 속성을 격납하는 것을 허락하므로, **propedit** 나 **propset**를 사용하는 것 만으로는, 속성을 삭제하는 것이 할 수 없습니다. 예를 들어 이 커멘트는 기대되는 결과로는 **되지 않습니다**:

```
$ svn propset license ' ' calc/button.c
property 'license' set on 'calc/button.c'
$ svn proplist --verbose calc/button.c
Properties on 'calc/button.c':
  copyright : (c) 2003 Red-Bean Software
  license :
$
```

속성의 삭제에는 **propdel** 커멘트를 사용할 필요가 있습니다. 구문은 다른 속성 관련 커멘드와 잘 닮았습니다:

```
$ svn propdel license calc/button.c
property 'license' deleted from ' '.
$ svn proplist --verbose calc/button.c
Properties on 'calc/button.c':
  copyright : (c) 2003 Red-Bean Software
$
```

이것으로, 속성 관련 **svn** 서브 커멘드의 모두에게 대해 설명했으므로, 일상적인 Subversion 워크플로우에, 속성의 변경 하지만 어떠한 영향을 줄까를 봅시다. 전에 지적한 것처럼 파일과 디렉토리의 속성은, 보통 파일의 내용과 같이, 버전화 됩니다. 결과적으로, Subversion은 다른 사람이 한 수정점을 자기 자신 위에 merge 할 수가 있습니다. 물론 통상의 merge와 같이, 잘될지도 모르지않고, 충돌할지도 모릅니다.

리비전 속성의 merge

버전화 되지 않는 리비전 속성을 기억하고 있을까요? **svn** 프로그램으로 이러한 속성도 수정 할 수 있습니다. 단지 **--revprop** 인수를 추가해, 수정하고 싶다 속성이 있는 리비전을 지정해 주세요. 이 경우 리비전은 글로벌이므로 수정하려고 하는 리비전 속성의 저장소(repository)로부터 체크아웃 한 작업 카피에 있는 한, 패스를 지정할 필요는 있어 선. [\[3\]](#)

```
$ svn propset svn:log '* button.c: Fix a compiler warning. ' -r11 --revprop
property 'svn:log' set on repository revision '11'
$
```

이러한 버전화 되지 않는 수정 권한은, 저장소(repository) 관리자에 의해 명시적에게 주지 않으면 안됩니다(>참조). 속성은 버전화 되지 않는다 의로 주의해 편집 없으면 정보를 잃는 위험이 있습니다. 저장소(repository) 관리자는 이 위험으로부터 지키기 위한 구조를 만들 수 있습니다만, 디폴트에서는 버전화 되지 않는 속성의 변경은 불가가 되어 있습니다.

그리고 파일의 내용의 경우와 같이, 속성의 변경은 로컬인 수정 밖에 지나지 않고, **svn commit**로 저장소(repository)에 커밋해 처음 수정이 확정합니다. 변경은 역시 간단하게 취소한다 일도 할 수 있는 **svn revert** 커멘드는 파일이나 디렉토리를 편집전 상태에 되돌려, 그 내용, 속성, 등 모두에 대해서도 그렇습니다. 게다가 **svn status** 나 **svn diff** 커멘드를 사용해, 파일이나 디렉토리 속성 상태에 대해 유용한 정보를 받을 수가 있습니다.

```
$ svn status calc/button.c
M      calc/button.c
```



```
$ svn diff calc/button.c
Property changes on: calc/button.c
```

```
Name: copyright
+ (c) 2003 Red-Bean Software
```

```
$
```

status 서브 커멘드가M(을)를 최초의 칼럼이 아니고, 두번째의 칼럼에 표시하는데 주의입니다. 이것은,calc/button.c의 속성을 수정했지만, 파일의 내용은 변경하고 있지 않는 것을 나타내고 있습니다. 속성도 내용도 변경하면,M 는, 최초의 칼럼 에도 두번째의 칼럼에도 표시 됩니다. (>참조).

속성의 충돌

파일의 내용과 같이, 로컬의 속성 변경은 다른 사람의 커밋에 받수정과 충돌할지도 모릅니다. 작업 카피를 갱신해, 자신 자신의 수정을 부수어 버리는 것 같은 버전화 된 리소스의 속성의 변경을 받아들였을 경우, Subversion은 그 리소스가 충돌의 상태에 있는 것을 보고합니다.

```
% svn update calc
M calc/Makefile.in
C calc/button.c
Updated to revision 143.
$
```

Subversion은 또, 충돌한 리소스가 있는 같은 디렉토리에 .prej 의 확장자(extension)를 가진 파일 (을)를 만듭니다. 이 안에, 충돌의 상세가 쓰여져 있습니다. 이 파일 의 내용을 확인해, 충돌을 어떻게 해소할까를 결정해 주세요. 충돌이 해소될 때까지,**svn status**의 출력의 두번째의 컬럼에C 가 표시되어, 로컬 수정 (을)를 커밋하려고 해도 실패하는 것을 나타냅니다.

```
$ svn status calc
C calc/button.c
? calc/button.c.prej
$ cat calc/button.c.prej
prop 'linecount': user set to '1256', but update set to '1301'.
$
```

속성의 충돌을 해소하려면, 충돌한 속성이 올바른 값을 포함하고 있을까 제발을 확인하고 나서,**svn resolved** 커멘드를 사용해 Subversion에 손으로 문제를 해소한 것을 보고해 주세요.

Subversion가 현재의 속성의 차이를 표시하는 표준적이 아닌 방법으로 눈치 채었을지도 모릅니다. **svn diff** 를 실행해, 출력을 패치 파일을 만들기 위해서(때문에) 리디렉트 할 수가 있습니다. **patch** 프로그램은 속성에 대한 패치를 무시합니다일반적으로 그것은 이해할 수 없는 쓰레기를 모두 무시합니다. 이것은 불행하게도 **svn diff**로 생성된 패치를 완전하게 적용하려면, 속성의 수정에 대해서는 손으로 적용하지 않으면 안 된다고 하는 것을 의미합니다.

본 것처럼, 속성의 수정은 전형적인 Subversion의 워크플로우에는 별로 큰 영향을 주지 않습니다. 작업 카피를 갱신해, 파일과 디렉토리의 상태를 체크해, 자신이 한 변경점에 대해 보고해, 그러한 수정점을 저장소(repository)에 커밋한다고 하는 일반적인 패턴은 속성의 존재나 비존재 (와)과는 완전하게 무관계합니다. **svn**프로그램에는 몇개의 추가의 서브 커멘드가 있어, 실제로 속성 변경할 수가 있습니다. 그러나, 그것은, 속성 관련 커멘드의 눈에 보이는 유일한 비대상성입니다.

1.2.3. 특수한 속성

Subversion은 속성에 대해 특별한 폴리시를 갖고 있지 않습니다어떠한 목적으로도 사용할 수가 있습니다. Subversion은,svn:라고 한다 프리픽스가 붙은 속성명을 사용하는 것을 금지하고 있을 뿐입니다. 이것이, Subversion 자신이 사용하는 속성의 이름 공간입니다. 실제,

Subversion은, 파일이나 디렉토리에 특수한 효과를 미 것 같은 어떤 종류의 속성을 정의하고 있습니다. 이 마디에서는 이 신비를 설명해, 어떻게 이것들 특수한 속성이, 당신의 Subversion 라이프를 조금 편하게 할까에 임해서 설명합니다.

1.2.3.1. svn:executable

svn:executable 속성은 반 자동적인 방식으로 버전 관리되고 있는 파일의 파일 시스템상의 실행 권한을 제어하는데 사용됩니다. 이 속성은 속성치를 굳이 정의 하지 않습니다단지 속성명이 존재하고 있으면, Subversion에 의해 실행 비트 하지만 보존됩니다. 이 속성을 삭제하면(자), 실행 비트의 전제어는 operating system에 되돌려집니다.

많은 operating system상에서, 커멘드로서 파일을 실행할 수 있을지 여떨지는 실행 비트의 존재에 의해 지배되고 있습니다. 이 비트는 보통, 디폴트에서는 무효가 되고 있어, 필요에 따라서 유지 하지만 명시적으로 유효하게 해 줄 필요가 있습니다. 작업 카피중에서는, 새롭다 파일이 항상 만들어져 그 한편, 갱신 처리를 통해서 존재하고 있다 파일의 새로운 버전을 받습니다. 이것은, 어느 파일의 실행 비트를 유효하게 하고 나서 작업 카피를 갱신했을 경우, 만약 갱신 처리의 일환으로서 그 파일이 변경되었을 때에 그 실행 비트는 무효가 되어 끝낼 가능성이 있다고 하는 것입니다. 거기서 Subversion은 svn:executable 자산을, 실행 비트를 유효하게 계속 하기 위해서(때문에) 준비해 있습니다.

이 속성은 FAT32나 NTFS와 같이 실행권한비트의 개념을 가지지 않는 파일 시스템 위에서는 아무 효과도 없습니다. [\[4\]](#) 또, 그것은 정의된 값을 가지지 않습니다만, Subversion은 이 속성이 설정된다 라고 강제적으로 그 값을*로 합니다. 마지막으로, 이 속성은 파일 에 대해서만 유효해, 디렉토리에 대해서는 의미를 가지지 않습니다.

1.2.3.2. svn:mime-type

svn:mime-type 속성은, Subversion에서는 여러가지 목적으로 사용됩니다. 파일 자신의 Multipurpose Internet Mail Extensions (MIME) 상의 분류의 기억 장소에서 있는 것과 동시에, 이 속성의 값은 Subversion 자신의 몇개의 동작 모드를 결정합니다.

예를 들어, 파일의svn:mime-type 속성이 비텍스트 MIME 타입인 경우(예외는 있든, 일반적으로는,text/ 이외로 시작되는 것 같은 경우), Subversion은 파일 내용은 바이너리이라고 가정 합니다즉, 가독이 아닌 . 이 이점의 하나는, Subversion 하지만, 작업 카피 갱신시에, 서버로부터 받는 변경점을, 문맥에 의존해 행 단위에 merge 하는 기능을 제공하는 것입니다. 그러나, 바이너리 데이터라고 믿을 수 있어 있는 파일에 대해서는"행"과 같은 개념은 전혀 없습니다. 그래서, 이러한 파일에 대해서는, Subversion은 갱신시에 문맥 merge를 실행하려고는 하지 않습니다. 그 대신, 바이너리의 작업 카피 파일을 수정해, 그것이 갱신되는 경우는 언제라도, 당신의 파일은 . orig 확장자(extension)를 붙인 형태로 명칭 변경되어 그리고 Subversion은 갱신으로 받는 변경을 포함하지만, 당신 자신의 로컬인 수정은 포함하지 않은 새로운 작업 카피 파일을, 원래의 이름으로 보존합니다. 이 행동은, 문맥 merge 할 수 없는 파일에 문맥 merge를 실행하려고 하는 잘못된 의도로부터 유저를 지키기 (위해)때문입니다.

Subversion은, 유저에 대해서, 바이노리필드 검출 알고리즘을 실행하는 것으로 유저를 돕습니다. 이것은**svn import** (와)**svn add** 서브 커멘드로 실행됩니다. 이러한 서브 커멘드는 파일이"바이너리 같은가"어떤가 (을)를 추측하는 발견적인 방법을 사용해, 그렇다고 생각되는 모든 파일의 svn:mime-type 속성을 application/octet-stream 로 설정합니다. ("이것은 아라바이트의 줄입니다" 라고 하는 일반적인 MIME 타입입니다) 만약 Subversion가 잘못된 추측을 하거나svn:mime-type 속성을 좀 더 정확하게 설정하고 싶은 경우는아마,image/png 라든지 application/x-shockwave-flash라든지언제라도 속성을 손으로 삭제하거나 수정할 수가 있습니다.

마지막으로, 만약svn:mime-type속성이 설정되어 있으면(자), Subversion의 Apache 모듈은 GET 요구에 응답할 때, HTTP 헤더의 Content-type:에 이 값을 사용합니다. 이것은 브라우저를 사용해 저장소(repository)를 조사할 때, 그 파일을 어떻게 표시하면 좋은가의 중요한 단서가 됩니다.

1.2.3.3. svn:ignore

svn:ignore 속성은 어떤 종류의 Subversion 조작이 무시한다 파일 패턴의 리스트를 포함하고 있습니다. 아마 가장 자주(잘) 이용된다 특수 속성으로,global-ignores 실행시 설정 옵션과 함께 이용됩니다. (>참조). 그것을 사용해, 버전화되어 있지 않은 파일과 디렉토리를 **svn status**와 같은 커멘드의 대상으로부터 제외합니다.

svn:ignore속성의 배후에 있는 이유는 간단하게 설명할 수 있습니다. Subversion은, 작업

커피 디렉토리에 있는 모든 파일과 서브 디렉토리 하지만 버전 관리하에 있다고는 가정하지 않습니다. 리소스는 **svn add** (을)를 사용해 명시적으로 Subversion 관리하에 둘 필요가 있습니다. 결과적으로 자주 작업 카피중이 많은 리소스가 버전 관리하에 없는 것이 있습니다.

svn status 커멘트는 출력의 일부로서 작업 카피에 어느 버전화되어 있지 않은 파일이나 서브디렉트리를, global-ignores 옵션(혹은 그 편입의 디폴트치에 의하고)에 의해, 아직 필터되어 있지 않은 것에 대한 보고 표시합니다. 이와 같이 행동하는 것은, 유저가, 어느 리소스를 버전 관리하에 추가하는 것을 잊었을 때에, 그것을 알 수 있도록(듯이) 하기 (위해)때문입니다.

그러나 Subversion은 무시해야 할 모든 리소스의 이름을 추측할 수 있다 (뜻)이유가 아닙니다. 게다가 매우 자주(잘), 특정의 저장소(repository)의, **모든** 작업 카피중에서 무시하고 싶은 것이 있기도 합니다. 그 저장소(repository)의 모든 유저에게, 각각의 실행시 설정 area에 특정의 리소스 패턴을 추가하도록(듯이) 강요하는 것은, 부담이 된다 만이 아니고, 유저가 체크아웃 한 다른 작업 카피의 설정에 의해 망가져 버리는 위험이 있습니다.

이것을 해결하려면, 어느 디렉토리에 나타날지도 모르는 리소스를 구별해 무시할 수 있는 것 같은 패턴을, 디렉토리 자체에 보존하는 것입니다. 버전화 되지 않는 리소스가 좋게 있는 예로, 기본적으로는 디렉토리 마다 독특하지만, 나타나는 일이 있는 것은, 프로그램의 컴파일로부터의 출력 등이 있습니다. 혹은이 본자신을 예를 들면 HTML, PDF, PostScript 파일 등으로, 이것들은 있는 DocBook XML 입력 파일을, 좀 더 읽기 쉬운 출력 형식으로 변환한 결과 생성되는 것입니다.

CVS 유저에게 있어서의 무시 패턴

Subversion의 `svn:ignore` 속성은 CVS 의 `.cvsignore` 파일에 구문도 기능도 매우 잘 닮았습니다. 실제조사 있고, CVS 작업 카피를 Subversion (으)로 이행하는 경우, `.cvsignore` 의 무시 패턴을 직접 **svn propset** 커멘트 의 입력 파일로 할 수가 있습니다:

```
$ svn propset svn:ignore -F .cvsignore .
property 'svn:ignore' set on '.'
$
```

그러나, CVS 와 Subversion가 패턴을 무시하는 방법에는 몇개의 차이가 있습니다. 두 개의 시스템은 무시 패턴을 다른 타이밍에 이용해, 무시 패턴이 적용되는 대상 정원않고일 까 차이점이 있습니다. 게다가 Subversion은 무시 패턴이 아무것도 없는 상태에 되돌리기 위한 ! 패턴은 사용할 수가 없습니다.

이러한 의미로, `svn:ignore` 속성이 해결법으로 됩니다. 그 값은 파일 패턴의 복수행의 모임으로, 일행에 하나의 패턴을 씁니다. 속성은, 패턴을 적용하고 싶으면 생각하는 디렉토리로 설정됩니다. [\[5\]](#) 예를 들어, **svn status**로부터의 이하의 출력이 있었다고 합니다:

```
$ svn status calc
M      calc/button.c
?      calc/calculator
?      calc/data.c
?      calc/debug_log
?      calc/debug_log. 1
?      calc/debug_log. 2. gz
?      calc/debug_log. 3. gz
```

이 예에서는, `button.c`에 대한데인가의 속성의 변경을 했습니다만, 작업 카피중에는 몇개의 버전 관리해 없는 파일도 있어, 그것은 이 경우, 원시 코드로부터 컴파일 한 `calculator` 프로그램, `data.c`라는 이름의 원시 코드, 그리고, 디버그 출력의 로그 파일입니다. 이것으로, 빌드 시스템은 항상 `calculator`를 생성하는 것을 알고 있습니다. [\[6\]](#) 그리고, 테스트 프로그램은 항상 이러한 디버그 로그 파일을 남기는 일도 알고 있습니다. 이러한 사실은 당신의 것 만이 아니고, 어느 작업 카피에 있어서도 올바른 일입니다. 그리고 **svn status** (을)를 실행할 때마다 이러한 파일을 보는 것에 흥미가 있는 것은 아닌 것도 알고 있습니다. 그래서, **svn propedit svn:ignore calc** (을)를 사용해 몇개의 무시 패턴을 `calc` 디렉토리에 추가합니다. 예를 들어 `svn:ignore` 속성의 새롭다 값으로서 이하를 추가할지도 모릅니다:

```
calculator
```

```
debug_log*
```

이 속성을 추가하면(자),calc디렉토리상에 로컬인 속성 변경을 손에 넣을 수가 있습니다. 그러나,svn status 출력에 대해 무엇이 바뀌었는지를 주의해 주세요:

```
$ svn status
M      calc
M      calc/button.c
?      calc/data.c
```

이것으로, 보고 싶지 않은 파일이 출력으로부터 전부 사라졌습니다. 물론 이러한 파일은 아직 작업 카피에 있습니다. Subversion 는 그것이 존재하고 있어, 버전 관리하에 없는 것에 도착해 (은)는 아무것도 말하지 않습니다. 이것으로, 표시로부터 시시한 파일을 전부 없앤다면, 좀 더 주의할 필요가 있는 아이템에 대해서는 그대로 하는 예를 들어, 버전 관리하에 추가하는 것을 잊은 원시 코드 파일 등은, 여전히 표시됩니다.

무시하는 파일을 보고 싶은 경우는, Subversion에 **--no-ignore** 옵션을 건네줄 수가 있습니다:

```
$ svn status --no-ignore
M      calc/button.c
I      calc/calculator
?      calc/data.c
I      calc/debug_log
I      calc/debug_log. 1
I      calc/debug_log. 2. gz
I      calc/debug_log. 3. gz
```

1.2.3.4. svn:keywords

Subversion 는 *키워드*를 파일 자신의 내용과 해 옮겨놓는 기능이 있습니다. 키워드란, 버전화 되었다 파일에 대한 도움이 되는 작고 동적인 정보입니다. 키워드는 일반적으로 파일이 마지막에 수정되었을 때 각에 대한 정보를 나타내고 있습니다. 이 정보는 파일이 변경될 때마다 바뀌어, 한층 더 중요한 일에는 파일이 변경된 **직후**에 바뀌므로, 그것은 데이터를 완전하게 최신 상태에 유지하는 것은, 버전 관리 시스템 이외 목과 같은 수단에 있어서도 귀찮은 것입니다. 편집한 인간에게 맡기면, 그 정보는 필연적으로 낡아집니다.

예를 들어, 수정된 마지막 일자를 표시하고 싶다고 생각하고 있는 문서가 있다고 합니다. 당신은, 그 문서의 모든 저자에게, 변경점을 커밋하기 직전에, 마지막에 변경된 자국을 나타내는, 문서의 일부를 조금 바꾸는 작업을 강요할 필요가 있습니다. 그러나, 조만간에, 누군가 이것을 잊는 사람이 나오겠지요. 그렇게 하는 대신에, 단지 Subversion에 대해서 LastChangedDate 키워드에 대해서 키워드 치환을 실행하도록 부탁드립니다. 당신은 문서중의 *keyword anchor* 를 두는 것 그리고 키워드가 삽입된, 파일중의 임의의 장소를 제어할 수가 있습니다. 이 앵커 캐릭터 라인은, 단지 \$ *KeywordName* \$와 같이 서식인가 되었다 캐릭터 라인입니다.

Subversion 는, 치환가능한 키워드의 리스트를 정의하고 있습니다. 그 리스트는, 이하의 다섯 개의 키워드로, 그 몇개인가에 대해서는 좀 더 짧은 별명을 사용할 수도 있습니다:

LastChangedDate

이 키워드는 파일이 저장소(repository)중에서 수정된 마지막 시각 (을)를 나타내, \$LastChangedDate: 2002-07-22 21:42:37 -0700 (Mon, 22 Jul 2002) \$(와)과 같은 것입니다. 이것은 Date와 생략 할 수도 있습니다.

LastChangedRevision

이 키워드는, 파일이 저장소(repository)로 변경된 마지막 리비전 (을)를 나타내, \$LastChangedRevision: 144 \$(와)과 같은 것입니다. 이것은 Rev와 생략 할 수도 있습니다.

LastChangedBy

이 키워드는 저장소(repository)중의 이 파일을 마지막으로 변경한 유저 (을)를 나타내, \$LastChangedBy: harry \$(와)과 같은 것입니다. 이것은 Author와 생략 할 수도 있습니다.

HeadURL

이 키워드는 저장소(repository)중의 파일의 마지막 버전 에 대한 완전한 URL를 나타내, \$HeadURL: http://svn.collab.net/repos/trunk/README \$ (와)과 같은 것입니다. 이것은URL와 생략 한다 일도 할 수 있습니다.

Id

이 키워드는, 다른 키워드의 압축된 편성입니다. 그 치환은, \$Id: calc.c 148 2002-07-28 21:30:43Z sally \$(와)과 같은 것으로, 파일 calc.c 가 마지막에 변경된 것은 리비전 148 으로, 시간은 July 28, 2002 의 밤, 변경한 사람은, sally인 것을 의미하고 있습니다.

단지 키워드 앵커 텍스트를 파일에 덧붙여도 아무것도 일어나지 않습니다. Subversion 는 명시적에 그렇게 하는 것을 요구하지 않으면, 결코 텍스트 치환을 하려고는 하지 않습니다. 결국, 당신은, 어떻게 해 키워드를 사용할까에 대한 문서를 [\[7\]](#) 쓰면, 치환되지 않는 키워드 앵커의 훌륭한 예자신이 Subversion에 따라서 치환되어 버리는 것을 바라지 않을 것입니다.

Subversion가 특정의 파일 위에서 키워드를 치환하는지 어떤지를 설정 하기 위해서, 속성 관련의 서브 커멘드로 돌아옵니다. svn:keywords 속성은, 버전 파일로 설정 되었을 경우는, 그 파일의 어느 키워드가 치환될까의 제어를 합니다. 그 값은, 공백에서 단락지어진 키워드 명칭이나 별명의 리스트로, 전에 쓴 테이블안에 있는 것의 어떤 것인가가 됩니다.

예를 들어,weather.txt 라는 이름의 버전 관리되고 있는 파일이 있어, 이하 같다고 합니다:

```
Here is the latest report from the front lines.
$LastChangedDate$
$Rev$
Cumulus clouds are appearing more frequently as summer approaches.
```

svn:keywords 속성이 파일로 설정되어 있지 않으면 Subversion은 굳이 특별한 (일)것은 하지 않습니다. 그런데, LastChangedDate 키워드의 치환을 유효하게 해 봅시다.

```
$ svn propset svn:keywords "LastChangedDate Author" weather.txt
property 'svn:keywords' set on 'weather.txt'
$
```

이것으로,weather.txt 의 로컬 속성을 변경 섬 했다. 그 파일의 내용에는 아무 변화도 없을 것입니다(속성을 설정 하기 전으로 변경하고 있지 않으면). 파일은 키워드 앵커 Rev키워드를 포함하고 있었다고 합니다. 우리는 이 키워드를 아직 속성치로서 설정해 있지 않습니다. Subversion은 파일 에 존재하지 않는 키워드를 치환하는 요구를 무시하고, svn:keywords 속성치에 존재하지 않는 키워드를 치환하는 일도 없습니다.

키워드와 가짜의 차분

키워드 치환에 의한 눈에 보이는 형태의 결과로서, 이 기능을 사용해 버전 관리되고 있는 파일은, 적어도 한 개소, 내용이 달라 있다고 생각할지도 모릅니다. 이 개소란, 키워드 앵커가 치환된 장소입니다. 그러나, 실제로는 이런 일은 일어나지 않습니다. 로컬인 수정점을svn diff로 조사할 때와 svn commit로 로컬 수정을 전송 하기 전에서는, Subversion 는 이전에 치환된 어떤 키워드도"한번 더 치환 할 것은 없습니다". 결과적으로, 저장소(repository)에 격납되었다 파일의 버전은 유저가 파일로 한 실제의 수정만이 포함되어 있습니다.

이 속성의 변경을 커밋한 직후, Subversion은 작업 파일을, 새롭다 치환 텍스트로 갱신합니다. 키워드 앵커 \$LastChangedDate\$를 보는 대신에, 치환 결과를 보게 되겠지요. 이 결과는 키워드의 이름을 포함해, 달러 기호 캐릭터(\$)로 묶여지고 있습니다. 그리고 말한 것처럼, Rev 는 설정하고 있지 않았기 때문에, 치환되지 않았습니다.

```
Here is the latest report from the front lines.
$LastChangedDate: 2002-07-22 21:42:37 -0700 (Mon, 22 Jul 2002) $
$Rev$
Cumulus clouds are appearing more frequently as summer approaches.
```

만약 누군가별의 사람이 `weather.txt`로 변경점을 커밋 하면, 파일의 카피는 전과 같은 치환된 키워드치를 계속 표시한다. 그렇지않업 카피를 갱신할 때까지는. 그 때, `weather.txt` 파일의 키워드는 그 파일을 커밋한 제일 마지막 상태를 반영하는 정보로 치환되겠지요.

1.2.3.5. svn:eol-style

버전 파일의 `svn:mime-type` 속성으로 지정하므로 없으면, Subversion은 파일은 가독인 데이터가 포함되어 있다 (와)과 과정 합니다. 일반적으로, Subversion은 그 파일에 대한 문맥 차분을 보고할 수가 있는지 어떤지를 결정하기 위해서(때문에)만 이용합니다. 그레 없으면, Subversion에 있어, 아라바이트는 단순한 아라바이트로 밖에 없습니다.

이것은, 디폴트에서는 Subversion은 당신의 파일이 이용하고 있다 *행단 (EOL) 마커*의 종류에 주의를 향하지 않는 것을 의미합니다. 불행하게도, 다른 operating system는 파일의 그것의 줄 끝을 나타내는데 다른 토큰을 사용합니다. 예를 들어, 보통 Windows 플랫폼의 소프트웨어에 의해 사용되는 줄 끝 토큰은 ASCII 제어 캐릭터의 조가 됩니다.왕복대 리턴(CR)과 라인 피드(LF)입니다. 그러나 Unix에서는 단지 LF 캐릭터를 사용해 줄 끝을 표현합니다.

이러한 operating system 위의 다양한 툴의 모든 것이 자신이 실행되고 있는 operating system의 *원래의 줄 끝 스타일 ending style* (와)과는 다른 형식의 줄 끝을 포함하고 있는 것 같은 파일을 이해할 수가 있다 (뜻)이유가 아닙니다. 자주 있는 결과적으로는, Unix의 프로그램은 Windows의 파일에 있는 CR 캐릭터를 통상의 캐릭터 (보통, ^M와 같이 표시합니다) (으)로서 취급해, Windows의 프로그램은 Unix 파일의 모든 행을 하나의 거대한 행으로서 연결해 버립니다만, 이것은 줄 끝을 나타내는 왕복대 리턴 - 라인 피드 캐릭터(혹은 CRLF)의 편성을 발견될 수 있는들 없기 때문에입니다.

이, 다른 EOL 마커에 관한 민감함은, 다른 오퍼레이팅(operating) 시스템간에 파일을 공유하려고 하는 사람을 주물러들 시킵니다. 예를 들어, 원시 코드 파일과 이 파일을 Windows에서도 Unix에서도 편집하는 개발자를 상상해 보세요. 만약 모든 개발자 하지만 항상 줄 끝을 보존하는 것 같은 툴을 사용한다면 문제는 일어나지 않습니다.

그러나, 실제로는, 많은 혼히 있던 툴은 다른 EOL 마커의 파일 (을)를 올바르게 읽을 수가 없는지, 파일이 보존될 때, 줄 끝을 그 operating system 고유의 것으로 변환해 버릴까 합니다. 만약 만약 개발자에게 최초의 일이 일어나면(자), 그는 외부의 변환 유틸리티 (**dos2unix**나, 그것과 페어가 되었다 **unix2dos**)를 사용해 파일 편집의 사전 처리를 하지 않으면 되지 않습니다. 그리고의 경우에는 굳이 특별한 준비는 필요 없습니다. 그러나 어느 쪽의 경우에서도, 모든 행이, 최초의 것과 달리 끝납니다. 변경을 커밋하기 전에, 유저에게는 2로 우리의 선택이 있습니다. 편집하기 전의 줄 끝 스타일과 같은 스타일이 되도록(듯이) 변환 유틸리티 (을)를 사용해 수정한 파일을 보존하는지, 단지 그 파일을 커밋할까 입니다이 경우, 줄 끝은 새로운 EOL 마커가 다합니다.

이러한 시나리오의 결과는 시간의 헛됨과 커밋된 파일에 대한 불필요한 수정이 됩니다. 시간의 헛됨은 그 만큼으로 충분한 고통입니다. 그러나, 커밋이 파일의 모든 행을 변경한다면, 이것은, 정말로 수정되었다 의는 어느 행인가를 결정하는 작업을 매우 복잡한 것으로 합니다. 버그의 수정은 도대체 어느 행 (로) 이루어졌는지? 어느 행으로 구문 에러가 있었는지?

이 문제의 해결은, `svn:eol-style` 속성입니다. 이 속성이 올바른 값으로 설정되면, Subversion은 그것을 사용해, 어떠한 특수한 처리가 파일에 필요하고, 그 처리를 하면 파일의 줄 끝 스타일이, 다른 operating 시스템으로부터의 커밋에 의해, 푸드득푸드득 변화하거나 하지 않는지, 를 결정합니다. 설정할 수 있는 값은:

native

이것은, 파일이, Subversion가 실행되고 있는 오퍼레이팅(operating) 시스템의 본래의 EOL 마커를 포함하도록(듯이) 합니다. 바꾸어 말하면(자) 만약 Windows상의 유저가 작업 카피를 체크아웃 해, 거기에는 `svn:eol-style` 속성이 `native` (으)로 설정된 파일이 있는 경우, 그 파일은 CRLF EOL 마커를 포함한다고 하는 것입니다. 반대로 Unix 유저가 작업 카피를 체크아웃 해, 거기에, 그 같은 파일이 있었을 경우는, 파일의 카피에는 LF EOL 마커가 포함되게 됩니다.

Subversion은 실제로는 저장소(repository)에 파일을 격납할 경우에는, operating system에는 따르지 않고, 정규화된 LF EOL 마커를 사용합니다. 이것은 기본적으로 유저에게는 의식하지 않아도 좋게 되어 있는 것입니다만.

CRLF

이것은 사용하고 있는 operating system에 의하지 않고, 파일의 EOL 마커에 CRLF의 줄을 사용합니다.

LF

이것은 사용하고 있는 operating system에 의하지 않고, 파일의 EOL 마커 에LF 캐릭터를 사용합니다.

CR

이것은 사용하고 있는 operating system에 의하지 않고, 파일의 EOL 마커 에CR 캐릭터를 사용합니다. 이 줄 끝 스타일은 그만큼 일반적이지는 않습니다. 그것은 낡은 Macintosh 플랫폼에서 이용되고 있었습니다. (게다가에서는 Subversion은 실행한다 것마저 가능하지 않습니다만)

1.2.3.6. svn:externals

svn:externals 속성은 하나 이상의 체크아웃 된 Subversion 작업 카피로 버전 관리된 디렉토리를 만들기 위한 인스트럭션을 포함하고 있습니다. 이 키워드에 관한보다 자세한 정보 (은)는>을 봐 주세요.

1.3. 외부 정의

가끔, 몇개의 다른 체크아웃에 의해, 하나의 작업 카피를 만드는 것이 편리한 일이 있습니다. 예를 들어, 저장소(repository)의 다른 장소에 있는 다른 서브 디렉토리를 갖고 싶다면, 저장소(repository) 자체가 별도로 다라고인합니다. 그러한 일을 손으로 설정하는 일도 물론 할 수 있는 **svn checkout** 를 사용해 네스트 한 작업 카피 구조와 같은 물건을 만드는 것입니다. 그러나, 이 레이아웃이 저장소(repository)을 사용하는 모든 사람에게 취해 중요하면, 다른 전원도 당신이 한 것과 같은 체크아웃 조작을 할 필요가 있습니다.

행운의 일로, Subversion은 *외부 정의*를 서포트 하고 있습니다. 외부 정의는, 로컬 디렉토리를 버전 관리되었다 리소스의 URL에 묶는 것입니다. Subversion에서는, svn:externals 속성을 사용해 외부 정의를 그룹으로 해 선언합니다. 이 속성은 버전 관리된 디렉토리로 설정되어 그 값은(속성이 설정된 버전 관리된 디렉토리 에 상대적인) 서브 디렉토리와 Subversion 저장소(repository) URL를 일행으로 한 복수행 테이블입니다.

```
$ svn propset svn:externals calc
third-party/sounds      http://sounds.red-bean.com/repos
third-party/skins       http://skins.red-bean.com/repositories/skinproj
third-party/skins/toolkit http://svn.red-bean.com/repos/skin-maker
```

svn:externals가 편리한 것은, 한번 버전 관리 아래의 디렉토리로 설정해 버리면, 그 디렉토리가 있는 작업 카피를 체크아웃 한 사람은만으로도 외부 정의의 혜택을 받을 수가 있다 곳입니다. 바꾸어 말하면(자), 누군가가 그러한 네스트 한 작업 카피의 체크아웃을 정의하면, 다른 사람은 아무도 거기에 붙어 고민하지 않아도 된다 그렇다고 하는 것입니다Subversion은, 원래의 작업 카피의 체크아웃 의 위도 외부 작업 카피를 체크아웃 할 수가 있습니다.

전의 외부 정의의 예를 봅시다. 누군가가calc 디렉토리의 작업 카피를 체크아웃 하면(자), Subversion은 그 외부 정의에 있는 아이템도 계속해 체크아웃 합니다.

```
$ svn checkout http://svn.example.com/repos/calc
A  calc
A  calc/Makefile
A  calc/integer.c
A  calc/button.c
Checked out revision 148.
```

```
Fetching external item into calc/third-party/sounds
A  calc/third-party/sounds/ding.ogg
A  calc/third-party/sounds/dong.ogg
A  calc/third-party/sounds/clang.ogg
```

```
A  calc/third-party/sounds/bang.ogg
A  calc/third-party/sounds/twang.ogg
Checked out revision 14.
```

```
Fetching external item into calc/third-party/skins
```


만약, 외부 정의를 변경할 필요가 있는 경우, 통상의 속성 변경 서브 커멘드를 사용해 줄 수 있습니다. `svn:externals` 속성에의 변경을 커밋할 때, Subversion은 다음의 **svn update**를 실행할 때의 변경된 외부 정의에 대해서 체크아웃 하는 아이템을 동기 합니다. 같은 것이, 다른 사람이 작업 카피를 갱신해, 당신이 변경한 외부 정의 (을)를 받을 때도 일어납니다.

1.4. 벤더 브랜치(branch)

개발중의 소프트의 경우가 전형적인 예입니다만, 버전 관리로 메인テナンス하고 있는 데이터가 자주 누군가와의 데이터에 밀접하게 관계하고 있든가, 혹은 의존하고 있다 일이 있습니다. 일반적으로 프로젝트로 요구되는 것은, 프로젝트의 안정성을 해치는 일 없이, 외부의 자원에 의해 제공되는 데이터를 할 수 있는 한 최신에 유지하는 것입니다. 이 생각은 항상 성립되지 않으면 안됩니다 그리고, 어느 그룹이 만든 정보는 다른 그룹에 의해 만들어진 것에 직접적인 영향을 준다고 하는 장소에서는 어디에서라도.

예를 들어, 소프트웨어 개발자가 서드 파티제의 프로그램 라이브러리를 사용했다 어플리케이션상에서 작업을 하고 있다고 합니다. Subversion은 Apache 휴대용 실행시 프로그램 라이브러리와 정확히 그러한 관계를 가지고 있습니다. (see >). Subversion의 원시 코드는 모든 가반성의 요구를 채우기 위해서(때문에), APR 프로그램 라이브러리에 의존하고 있습니다. Subversion의 개발의 초기의 단계에서는, 프로젝트는 APR 의 API의 변경을 매우 정확하게 뒤쫓고 있었습니다. 항상, 프로그램 라이브러리 코드의 거센 파도의, "최첨단"을 따라갔습니다. 지금에 와서는 APR도 Subversion도 개발이 빠짐벌 있고 (이)라고 왔으므로, Subversion은 자주(잘) 테스트되어 안정된 릴리스 상태에 있다 버전의 APR 프로그램 라이브러리 API와 동기를 잡고 있습니다.

만약 프로젝트가 다른 사람의 정보에 의존하고 있다면, 그 정보와 자신의 물건을 동기 시키기 위한 몇개의 방법이 있습니다. 제일 대단한 방법입니다만, 자신의 프로젝트의 모든 공헌자에 대해 구두 또는 문서로 수속을 전할 수가 있습니다. 프로젝트에 필요한 서드 파티의 정보의 특정의 버전을 확실히 손에 넣는 것을 전합니다. 만약 서드 파티의 정보가 Subversion 저장소(repository)로 관리 되고 있다면, Subversion의 외부 정의를 사용해, 효율적으로, 어느 장소에 두어지고 있는 자신의 작업 카피 디렉토리에, 그 정보의 특정의 버전 (을)를 "묶는"일이 생깁니다. (>참조).

그러나, 가끔 자신의 버전 콘트롤 시스템으로 서드 파티의 데이터 에 가세한 독자적인 변경을 관리하고 싶은 것도 있습니다. 소프트웨어 개발의 예로 돌아와 설명하면(자), 프로그래머는 자기 자신의 목적을 위해서(때문에), 서드 파티 의 프로그램 라이브러리로 변경을 더할 필요가 있을지도 모릅니다. 이러한 수정은 새로운 기능 추가이거나 버그 수정이거나 할지도 모르지 않습니다만, 그것은 서드 파티의 프로그램 라이브러리의 공식적인 릴리스의 일부가 되기까지 한계 관리해야할 것입니다. 혹은, 변경은 결코 프로그램 라이브러리 메인テナンス 담당에는 전해지지 않고, 소프트웨어 개발자의 특수한 요구에 맞는 것 같은 프로그램 라이브러리를 완성하기 위한 독자적인 수정점으로서 계속 쪽 남을지도 모릅니다.

여기서, 재미있는 상황에 직면합니다. 이 프로젝트는 서드 파티의 데이터에 대한 독자적인 수정을, 분리한 형태로 관리할 수가 있습니다만, 그것은, 패치 파일을 사용하거나 완전하게 만들어낸 별버전의 파일, 디렉토리가 될지도 모릅니다. 이러한 것은 곧바로 메인テナンス하는데 있어서 두통거리가 되므로, 이쪽의 독자적인 수정을 서드 파티 의 데이터에 적용해, 그러한 변경에 대한 필요한 재생성을, 서드 파티 데이터의 각각의 계속하는 버전을 바탕으로 해 실행하는 구조가 필요하게 됩니다.

이 문제에 대한 해결은, *벤더 브랜치(branch)*를 사용하는 것입니다. 벤더 브랜치(branch)는 서드 파티, 혹은 벤더 의 데이터에 의해 제공된 정보를 포함하고 있는, 이쪽의 버전 관리 시스템중에 둔 디렉토리 트리입니다. 각각의 버전의 벤더의 데이터로, 자신의 프로젝트에 수중에 넣으려고 생각하고 있는 것의 일을, *벤더 드롭*과 말합니다.

벤더 브랜치(branch)는 두 개의 열쇠가 되는 이점이 있습니다. 우선, 자신의 버전 콘트롤 시스템에, 현시점에서 서포트되고 있다 벤더 드롭을 격납하는 것에 의해, 프로젝트의 멤버는 올바른 버전의 벤더 데이터를 사용하고 있는지 어떤지의 걱정을 한다 필요가 없어집니다. 다음에, 데이터는 스스로의 Subversion 저장소(repository)에 있으므로, 독자적인 수정을 그대로 격납할 수가 있습니다. 스스로의 독자적인 수정으로 옮겨놓는 것 같은 자동화되었다(혹은 최악의 경우, 손으로 하는) 방법을 준비할 필요가 없어집니다.

1.4.1. 일반적인, 벤더 브랜치(branch)를 관리하는 방법

벤더 브랜치(branch)의 관리는 일반적으로는 이런 식으로 합니다. 최상정도 디렉토리를 만들어(/vendor와 같은 것) 거기에 벤더의 브랜치(branch)를 둡니다. 그리고 최상정도 디렉토리의 사브디렉트리에 서드 파티의 코드를 임포트 합니다. 그리고 그 사브디렉트리를, 적당한 장소에 있는, 자신의 주계 개발 브랜치(branch)에 카피합니다(예를 들어/trunk등) 로컬인 변경은 항상 주계 개발 브랜치(branch)에 대해서 행합니다. 뒤쫓고 있는 코드의 새로

운 릴리스마다, 그것을 베타 브랜치(branch)에 가지고 가, 변경점을/trunk에 merge 합니다. 그리고, 로컬의 변경과 베타의 변경동안의 충돌을 해소합니다.

아마, 예를 들면(자), 이 스텝을 확실히 할 수가 있을지도 알려지지 않습니다. 당신의 개발 팀이 서드 파티의 복잡한 수치계산 프로그램 라이브러리를 사용한 계산 프로그램을 만들고 있다고 합니다. 우선, 베타 브랜치(branch)의 초기 생성을 해, 그리고 최초의 베타 드롭(을)를 импорт 합니다.

```
$ svn import /path/to/libcomplex-1.0 W
http://svn.example.com/repos/calc/vendor/libcomplex/current W
-m 'importing initial 1.0 vendor drop'
```

이것으로, libcomplex의 원시 코드를/vendor/libcomplex/current 에 가지고 올 수가 있습니다. 이 버전에 태그 짓고 해, (>참조), 주계 개발 브랜치(branch)에 카피하면 거기에 독자적인 수정을 더할 수가 있게 됩니다.

```
$ svn copy http://svn.example.com/repos/calc/vendor/libcomplex/current W
http://svn.example.com/repos/calc/vendor/libcomplex/1.0 W
-m 'tagging libcomplex-1.0'
```

```
$ svn copy http://svn.example.com/repos/vendor/libcomplex/1.0 W
http://svn.example.com/repos/calc/libcomplex W
-m 'bringing libcomplex-1.0 into the main branch'
```

프로젝트의 주계의 브랜치(branch)를 체크아웃 합니다. 이것은 최초의 베타 드롭의 카피를 포함하고 있습니다. 그리고, libcomplex 코드의 수정에 들어갑니다. 이미 알고 있도록(듯이), 이것으로 수정된 libcomplex (은)는, 완전하게 계산 프로그램에 통합되고 있습니다. [\[8\]](#)

몇주간인가 쉼어, libcomplex의 개발자는 프로그램 라이브러리의 새로운 버전을 릴리스 했던 버전 1.1으로 합시다. 그것은 우리를 갖고 싶었다 몇개의 기능과 함수를 포함하고 있습니다. 그러나, 벌써 수중에 있는 버전에 대한 수정을 잃는 일 없이, 이 새롭다 버전에 업그레이드 하고 싶은 것입니다. 벌써 시사한 것처럼, 본질적으로 우리가 하지 않으면 안 되는 것은, libcomplex1.1의 카피 그리고 libcomplex1.0을 옮겨놓아 앞에 한 독자적인 수정을, 새로운 프로그램 라이브러리 버전에도 다시 적용하는 것입니다.

이 업그레이드를 하는데, 우리는 베타 브랜치(branch)의 카피(을)를 체크아웃 해, 현재의 버전을 새롭다 libcomplex1.1의 원시 코드로 옮겨놓습니다. 이 변경을 커밋했다 그리고, 우리의 current 브랜치(branch)는 새롭다 베타 드롭을 포함하고 있습니다. 이 버전을 태그 짓지워, 그리고 전에 태그 붙이고 한 버전과 새로운 현재의 버전의 사이의 차이(을)를 주계 개발 브랜치(branch)에 merge 합니다.

```
$ cd working-copies/calc
$ svn merge http://svn.example.com/repos/vendor/libcomplex/1.0 W
http://svn.example.com/repos/vendor/libcomplex/current W
libcomplex
# resolve all the conflicts between their changes and our changes
$ svn commit -m 'merging libcomplex-1.1 into the main branch'
```

간단한 경우라면 이 새로운 버전의 서드 파티 툴은, 파일과 디렉토리의 관점으로부터 보면(자), 전의 버전과 같이 보입니다. 다른 말투를 하면(자), 어느 libcomplex 원시 파일도 삭제되거나 명칭 변경되거나 다른 장소로 이동하거나 하고 있지 않습니다. 간단하게 말하면, 완전한 세계에서는, 우리의 수정은 새로운 프로그램 라이브러리의 버전에 예쁘게 적용되어 복잡한 일이나, 충돌은 일절 일어나지 않는다 그렇다고 하는 것입니다.

그러나, 모든 것이라는 것은 항상 단순하다라고는 인가 선. 실제조사 있고, 원시 코드는 소프트웨어의 릴리스간에 여기저기 움직이는 것이 보통입니다. 이것은 우리의 수정이 새로운 버전의 코드 그렇지만 올바르게 하고 있는 것을 확인하는 작업을 복잡하게 하고, 새로운 버전에서의 수정을 손으로 한번 더 할 필요가 있는 상황에, 간단하게 낙담해 버린다 일이 있습니다. Subversion가, 원시 파일의 히스토리에 대해 알아 있으면 프로그램 라이브러리의 새로운 버전에서의 merge의 스텝은 매우 단순하게 됩니다. 그러나, 우리는, Subversion에 원시 파일의 레이아웃이 베타 드롭간에 어떤 바람으로 바뀌었는지를 가르쳐 줄 책임이 있습니다.

1.4.2. svn-load-dirs.pl

몇개의 파일의 삭제, 추가, 이동이 있던 벤더 드롭은 서드 파티 데이터의 업그레이드의 순서를 복잡하게 합니다. 그래서 Subversion은 이 수속을 지원하기 위해서 **svn_load_dirs.pl** 스크립트를 준비해 있습니다. 이 스크립트는 일반적인 벤더 브랜치(branch)의 관리 수속으로 말한 것 같은 임포트의 스텝을 자동화해, 실수를 최소로 할 수가 있습니다. 서드 파티 데이터의 새로운 버전을 주게 개발 브랜치(branch)에 merge 한다 유익의 merge 커맨드를 사용할 책임은 아직 남아 있지만, **svn_load_dirs.pl**는 보다 빨리 간단하게 이 처리까지 도달하는 도움이 됩니다.

간단하게 말해, **svn_load_dirs.pl** 는 **svn import** 의 확장으로, 몇개의 중요한 특징을 가지고 있습니다:

- 언제라도, 이 프로그램을 실행해, 저장소(repository)에 있는 디렉토리를, 완전하게 거기에 일치한 외부 디렉토리에 가지고 가, 필요한 모든 추가, 삭제를 실행해, 한층 더 옵션으로 이동 처리도 실시합니다.
- 이 프로그램은, Subversion가 필요로 하는 중간적인 커밋간에 필요한 복잡한 일련의 처리를 주의 깊게 실행합니다 예를 들어 파일이나 디렉토리의 명칭 변경을 2회하기 전 등.
- 그것은, 옵션으로 나등 있고 임포트 된 디렉토리를 태그 짓고 합니다.
- 그것은 옵션으로, 정규 표현에 성냥 하는 파일과 디렉토리 에 임의의 속성을 추가합니다.

svn_load_dirs.pl 는 세 개의 필수 파라미터를 취합니다. 처음은 작업 대상이 되는 베이스가 되는 Subversion 디렉토리의 URL 입니다. 이 인수의 후에는 URL가 계속됩니다. 최초의 인수에 상대적인 형태로 벤더 드롭은 거기에 임포트 됩니다. 마지막에 3번째의 인수는 임포트 하는 로컬 디렉토리입니다. 전의 예를 사용하면(자), 전형적인 **svn_load_dirs.pl**의 실행은 이런 기분이 됩니다:

```
$ svn_load_dirs.pl http://svn.example.com/repos/calc/vendor/libcomplex W
current W
/path/to/libcomplex-1. 1
```

-t 옵션에 태그 명칭을 지정해, 새로운 벤더 드롭 (을)를 태그 짓고 하도록(듯이) **svn_load_dirs.pl** 에 지시하는 것이 할 수 있습니다.

```
$ svn_load_dirs.pl -t libcomplex-1. 1 W
http://svn.example.com/repos/calc/vendor/libcomplex W
current W
/path/to/libcomplex-1. 1
```

svn_load_dirs.pl를 실행할 때, 그것은 벌써 존재하고 있는 "현재의" 벤더 드롭의 내용을 조사해 그것을 지정된 새로운 벤더 드롭의 내용과 비교합니다. 간단한 경우, 다른 한쪽의 버전에 있어 이제 한편에는 없는 것 같은 파일은 없는 구로, 스크립트는 새로운 임포트를, 특히 문제 없게 실행합니다. 그러나, 만약, 버전간에 파일 레이아웃에 차이가 있는 경우, **svn_load_dirs.pl** 는 이 차이를 어떻게 해결할까 물어 옵니다. 예를 들어, libcomplex의 버전 1.0으로 math.c (이있)었던 파일은 libcomplex1. 1에서는 arithmetic.c 에 명칭 변경이 된 것을 알고 있는 것을 스크립트에 가르쳐 주는 것이 할 수 있습니다. 이동에 의해 설명할 수 없는 것 같은 차이점은, 통상의 추가와 삭제로서 다루어집니다.

그 스크립트는 또 저장소(repository)에 **추가되고 있다** 정규 표현에 성냥 하는 것 같은 파일과 디렉토리의 속성을 설정한다 위해(때문에), 다른 설정 파일을 받아들일 수가 있습니다. 이 설정 파일은 **svn_load_dirs.pl** 로 -p 옵션을 사용해 지정됩니다. 설정 파일의 각 행은 두 개에서 네 개의 공백에서 단락지어진 값입니다: 추가된 패스에 대해서 성냥 시키는 Perl 스타일의 정규 표현, 제어 키워드(break 또는 cont), 그리고, 옵션으로 속성명과 속성치가 옵니다.

```
W. png$          break  svn:mime-type  image/png
W. jpe? g$       break  svn:mime-type  image/jpeg
W. m3u$          cont   svn:mime-type  audio/x-mpegurl
W. m3u$          break  svn:eol-style  LF
. *              break  svn:eol-style  native
```

추가된 패스 마다, 정규 표현이 패스에 성냥 하는 것 같은 속성 변경 하지만 이 순서로 적용됩니다. 다만 제어의 지정이break (이)가 아닌 경우에 그렇게 됩니다(이것은 그 이상의 속성 변경은 이 패스에 행하지 않는 것을 의미하고 있습니다). 만약 제어 지정이cont continue의 생략형입니다만의 경우는 매칭 처리는 설정 파일의 다음의 행에 이어 갑니다.

정규 표현증의 모든 공백, 속성명, 속성치는 싱글 또는 더블 퀴츠 그리고 묶을 필요가 있습니다. 공백을 둘러싸기 위해서(때문에) 이용하고 있는 것은 아니다 퀴츠 캐릭터는 backslash 캐릭터(\)를 앞에 두고 붙인다 일로 이스케이프 할 수 있습니다. backslash는 설정 파일을 해석할 때 사나워지고 퀴츠 하프로, 정규 표현증에서 필요한 것 이외의 캐릭터에는 사용하지 말아 주세요.

Notes

- [1] 이 제의(신청)은, 대부분의 사람이 그렇다고 생각합니다만, Subversion에 일전도 지불하지 않은 사람들인 만큼 대하는 제의(신청)입니다.
- [2] 남은 것으로 디너는 어떠세요?
- [3] 커밋 로그중의, 스펠 미스, 문법 틀림, "시시한 미스" 는 아마--revprop 옵션 이용으로 1번 잘 일어나는 것입니다.
- [4] Windows의 파일 시스템은 파일 확장을 사용해 그것이 실행 파일이다 일을 나타냅니다. (. EXE, . BAT, . COM와 같은 확장자(extension)입니다)
- [5] 패턴은 그 디렉토리에만 제한됩니다서브 디렉토리에 재귀적으로 전해질 것은 없습니다.
- [6] 그것이 빌드 시스템의 핵심에서는?
- [7] 혹은, 그 책의 일절을
- [8] 그리고, 물론 당신이기 때문에, 버그도 완전하게 없어져 있다. 라고.

[EditText](#) | [Refresh](#) | [FindPage](#) | [DeletePage](#) | [LikePages](#) | [Tour](#) | [Keywords](#) |

W3C XHTML 1.0 W3C CSS W3C XHTML 1.0 W3C CSS W3C XHTML 1.0 W3C CSS

Last modified: 2004-01-05 16:18:12, Processing time: 0.0851 sec

일정관리프로그램

Google에서 전사용 공유 캘린더를 무료로 제 친구와 사진을 같이 볼까요? Picasa 웹앨범을
공. 지금 다운로드 하세요! 사용해 보세요

온라인 사진 앨범

친구와 사진을 같이 볼까요? Picasa 웹앨범을 사용해 보세요